

Blocco 11 - Performance di un DBMS per dashboard

EXPLAIN, indici e viste materializzate nel sistema ticketing

Relational Databases & SQL

30 min teoria + 90 min pratica

1. Il problema operativo

Quando una dashboard diventa lenta, il problema non è solo estetico: il DBMS sta facendo troppo lavoro.

Obiettivo del blocco

Problema

La dashboard Metabase esegue query sul DBMS. Se le query crescono o i dati aumentano, serve capire come PostgreSQL lavora e come aiutarlo con indici e oggetti precomputati.

Idea guida

Ottimizzare significa misurare, formulare un'ipotesi, modificare e misurare di nuovo.

- ▶ molte query sono ripetute spesso;
- ▶ i filtri sono simili: date, stato, priorità, regione;
- ▶ le aggregazioni leggono molte righe per produrre poche celle;
- ▶ Metabase può aggiornare più card insieme;
- ▶ il DBMS deve servire sia ingestione sia lettura.

- ▶ il collector inserisce nuovi ticket;
- ▶ la dashboard legge e aggrega;
- ▶ gli utenti applicano filtri;
- ▶ senza indici e viste adeguate il database fa scan e sort ripetuti;
- ▶ il problema cresce con il volume dati.

Analogia: magazzino e inventario

Caso esplicativo

Se ogni richiesta al cruscotto costringe a ricontare tutto il magazzino, il sistema non scala. Indici e viste materializzate sono strumenti per evitare lavoro ripetuto, ma hanno un costo di manutenzione.

2. Leggere il piano

Prima di creare un indice bisogna vedere cosa sta facendo PostgreSQL.

Query target

```
SET search_path TO ticketing;

EXPLAIN (ANALYZE, BUFFERS)
SELECT ticket_id, opened_at, priority, category, region, status
FROM support_tickets
WHERE status IN ('open', 'assigned', 'waiting_customer')
      AND opened_at >= TIMESTAMP '2026-05-01'
      AND opened_at <  TIMESTAMP '2026-05-14'
ORDER BY opened_at DESC, ticket_id DESC;
```

Cosa cercare nel piano

- ▶ Seq Scan: PostgreSQL legge tutta la tabella;
- ▶ Index Scan: usa un indice per localizzare righe;
- ▶ Sort: ordina dopo avere letto i dati;
- ▶ Rows Removed by Filter: righe scartate dopo la lettura;
- ▶ Buffers: pagine lette dalla cache o dal disco.

- ▶ il costo è una stima usata dall'ottimizzatore;
- ▶ il tempo reale arriva con `ANALYZE`;
- ▶ su dataset piccoli il piano può restare sequenziale;
- ▶ il metodo resta valido anche se il laboratorio ha pochi dati;
- ▶ la decisione va spiegata rispetto al workload atteso.

Definizione pratica

Un filtro è selettivo quando restituisce poche righe rispetto alla tabella. Un indice è più utile quando evita di leggere molte righe inutili.

Esempio: `priority = 'urgent'` può essere più selettivo di `status <> 'closed'` se pochi ticket sono urgenti.

3. Indici per dashboard

Un indice deve corrispondere a una query reale, non a una colonna vista a caso.

Pattern di filtro nel caso ticketing

- ▶ ticket aperti ordinati per data;
- ▶ ticket urgenti per regione;
- ▶ conteggi per stato e priorità;
- ▶ drill-down sui ticket non chiusi;
- ▶ ricerche per sorgente ed external id.

Indice parziale per ticket aperti

```
CREATE INDEX IF NOT EXISTS idx_ticketing_open_dashboard  
ON support_tickets (opened_at DESC, ticket_id DESC)  
WHERE status IN ('open', 'assigned', 'waiting_customer');
```

Perché indice parziale

- ▶ la dashboard operativa guarda soprattutto ticket aperti;
- ▶ i ticket chiusi non servono alla query di drill-down;
- ▶ l'indice è più piccolo di un indice su tutta la tabella;
- ▶ supporta filtro e ordinamento;
- ▶ va documentato perché dipende dalla definizione di “aperto”.

Indice composto per segmentazioni

```
CREATE INDEX IF NOT EXISTS idx_ticketing_priority_region  
ON support_tickets (priority, region, opened_at);
```

- ▶ le prime colonne devono riflettere i filtri più importanti;
- ▶ un indice composto non è un insieme generico di indici singoli;
- ▶ l'ordine influenza quali query vengono supportate bene;
- ▶ inserimenti e aggiornamenti diventano leggermente più costosi;
- ▶ per questo ogni indice deve avere una query target.

4. Precomputare metriche

Alcune dashboard leggono sempre gli stessi aggregati: si può materializzare.

Vista normale o materializzata

	Vista	Vista materializzata
Dati	calcolati al momento	salvati fisicamente
Aggiornamento	sempre attuale	richiede refresh
Costo lettura	può essere alto	spesso più basso
Uso	semantica stabile	dashboard pesanti o ripetute

Materialized view giornaliera

```
CREATE MATERIALIZED VIEW ticket_daily_metrics AS
SELECT opened_at::date AS day,
       priority,
       region,
       count(*) AS opened_tickets,
       count(*) FILTER (WHERE closed_at IS NOT NULL) AS closed_tickets,
       count(*) FILTER (WHERE
           COALESCE(closed_at, TIMESTAMP '2026-05-13 09:00:00') > sla_due_at
       ) AS sla_breached_tickets
FROM support_tickets
GROUP BY opened_at::date, priority, region;

CREATE INDEX idx_ticket_daily_metrics_day
ON ticket_daily_metrics(day, priority, region);
```

Query sulla materialized view

```
SELECT day,  
       sum(opened_tickets) AS opened_tickets,  
       sum(closed_tickets) AS closed_tickets,  
       sum(sla_breached_tickets) AS sla_breached_tickets  
FROM ticket_daily_metrics  
GROUP BY day  
ORDER BY day;
```

Aggiornamento

```
REFRESH MATERIALIZED VIEW ticket_daily_metrics;
```

Il refresh può essere manuale in laboratorio. In produzione sarebbe pianificato o orchestrato, e la frequenza dipenderebbe da quanto la dashboard deve essere aggiornata.

5. Ingestione e letture insieme

L'architettura deve sostenere sia nuovi dati sia dashboard.

- ▶ più indici migliorano alcune letture ma rallentano scritture;
- ▶ viste materializzate velocizzano dashboard ma possono essere non aggiornate;
- ▶ filtri Metabase non previsti possono aggirare gli indici;
- ▶ dataset piccoli non mostrano sempre benefici misurabili;
- ▶ il progetto deve dichiarare ipotesi di volume e frequenza.

- ▶ il volume raw cresce come previsto?
- ▶ ogni raw event ha un ticket curato?
- ▶ la dashboard legge viste e non tabelle raw?
- ▶ i piani delle query principali sono accettabili?
- ▶ gli indici creati hanno ancora una query che li giustifica?

Controllo raw vs curated

```
SELECT (SELECT count(*) FROM support_tickets_raw) AS raw_events,  
(SELECT count(*) FROM support_tickets) AS curated_tickets,  
(SELECT count(*)  
  FROM support_tickets_raw r  
  LEFT JOIN support_tickets t  
    ON t.external_ticket_id = r.external_ticket_id  
  WHERE t.ticket_id IS NULL) AS raw_without_curated_ticket;
```

6. Attività pratica

Misurare una query, proporre un indice e difendere la scelta.

Consegna

La tabella ticket cresce e la dashboard dei ticket aperti deve restare reattiva. Bisogna analizzare il piano della query, proporre un indice e motivare il trade-off.

- ▶ input: schema `ticketing` popolato;
- ▶ output: piano prima, indice, piano dopo, commento tecnico;
- ▶ vincolo: non creare indici senza query target;
- ▶ vincolo: includere almeno un controllo raw-curated;
- ▶ vincolo: discutere costo di scrittura e refresh.

Organizzazione dei 90 minuti

- ▶ 20 min lettura di EXPLAIN;
- ▶ 25 min query target e indice parziale;
- ▶ 20 min materialized view giornaliera;
- ▶ 15 min controllo raw-curved;
- ▶ 10 min discussione trade-off.

- ▶ una dashboard è un workload sul DBMS;
- ▶ indici e materialized view sono decisioni architetturali, non solo sintassi;
- ▶ ogni ottimizzazione va motivata con query target, piano e trade-off.